

응용 - D2 · 프로젝트 가이드

Coding Agent 실전: AI기반 TDD 마스터— 마방진 검증기

개념에서 코드까지, 추적성을 잃지 않는 하루 · C2C × Dual-Track TDD

Magic Square 4×4

계약 ID(INV·E·UC)

ARRR

추적성 매트릭스

이 과정을 관통하는 인과 사슬

① 목표

C2C — 요구·개념이 코드까지 추적 가능해야 한다

② 이유

추적성 확보 — 테스트 없는 코드는 어디가 왜 깨졌는지 모른다

③ 수단·사이클

Dual-Track TDD ≡ ARRR — 둘은 같은 것의 두 이름

④ 결과

추적 가능한 Clean Code — 테스트가 보호하는 깨끗한 코드

△ ARRR = Ask(RED) · Respond(GREEN) · Refine(REFACTOR) · Repeat(다음 계약 ID). 이 한 줄이 오늘의 전부.

하루 시간표 — 5개 세션 (08:30 ~ 17:30)

시간	세션	내용
08:55-10:15	세션 1	바이브 vs C2C · 3수준 사다리 · 미니 TDD 체험
10:30-11:45	세션 2	4계층 · 계약 ID · Dual-Track ≡ ARRR · 문제 정의
13:00-14:25	세션 3	Mom Test→계약(L0~L2) · R-G-I-O-PRD · 하네스 구축
14:40-16:05	세션 4	③~⑦ ARRR — RED·GREEN·Golden Master·REFACTOR·확장
16:15-17:30	세션 5 · 마무리	문서화 · /export · AARRR 계측 · KPT 회고

강의 40% + 실습 60% · 개발자 및 입문자 대상 · Magic Square 하나로 하루를 관통

Magic Square 4×4 — 하루를 하나로 잇는 예제

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

모든 행·열·대각선의 합 = 34

검증 라인 10개: R1~R4 · C1~C4 · D1 · D2

도메인

- 4×4 · 채운 칸 1~16 · 빈칸 0 · 마법상수 34
- 검증 대상 10선: R1~R4 · C1~C4 · D1·D2
- 목표 = 풀기 아님 → 어느 줄이 왜 틀렸는지 10초 안에
- 자동 풀이·채우기 = OOS(범위 밖)

```
validate_lines(grid) → {status, failed_lines}
```

7단계 = 논문 8단계 골격 + 13 STEP의 실행 · ③~⑥이 ARRR = C2C × Dual-Track TDD

SESSION 1

이론 — 바이브 코딩을 넘어

빠르게만 만들면 무엇이 무너지는가. C2C 문제의식과 AI 협업의 3수준 사다리.

08:55-10:15

이론

바이브 코딩 vs C2C 기반 코딩

바이브 코딩 (Vibe Coding)

- 프롬프트 → 완성 코드. 빠르지만 역추적 불가
- “정말 요구를 만족하나?” 판단 근거가 없음
- 고칠 때마다 불안 — 바닥은 낮추지만 천장이 없다

C2C 기반 코딩

- 모든 동작이 계약 ID(INV-*/E-*)에서 출발
- “ID 없는 동작은 구현하지 않는다”
- 테스트 ↔ 계약 ↔ 커밋으로 추적 — 천장을 지킨다

핵심 질문의 전환: “어떤 기법을 쓰는가” → “추적 고리가 끊긴 곳이 어디인가”

AI 협업의 3수준 사다리

① 프롬프트

이번 한 번의 지시

“이 실패 로그로 최소 구현해줘”

② 컨텍스트

지금 상황을 붙여 줌

@src/validate_lines.py · pytest 실패
로그

③ 하네스

영구 규칙·도구로 고정

GEMINI.md · settings.json · /커맨드 ·
테스트 루프

위로 갈수록 의도가 1회성 지시 → 영구 구조로 고정 · 반복 규율일수록 하네스로 올려라

SESSION 2

C2C와 Dual-Track TDD 프레임워크

계약이 코드까지 내려가는 4계층 · 추적의 못(계약 ID) · ARRR로 도는 두 트랙.

10:30-11:45

이론+미니실
습

C2C 4계층 검증 구조 — 위에서 아래로

계층	내용
① 제품 계약	진짜 문제에서 나온 불변식·에러·UI 계약 (INV/E/UC) + 증거 L0~L2
② 아키텍처(ECB)	Entity(도메인) · Boundary(입출력·에러) · Control · entity는 boundary import 금지
③ 계약 ID	추적의 못 — INV-*.E-*.UC- / 테스트 ID D-*.U-*
④ 테스트	계약 ID를 검증하는 실패 테스트(RED) → 최소 구현(GREEN)

위 계층이 아래 계층의 근거가 된다 — 어느 코드도 근거(계약) 없이 존재하지 못한다

계약 ID(추적의 못)와 추적성 매트릭스

계약 ID는 개념과 코드를 잇는 못입니다. 한 ID로 계약 ↔ 테스트 ↔ 구현 ↔ 커밋이 웨이면, 매트릭스에 빈칸이 없어집니다 = 정체불명 코드가 없다 = C2C 완성.

추적 사슬 한 줄

제품 계약 (INV-5) → 테스트 (D-LOC-01) → 구현 (locator.py, # [INV-5])
→ 커밋 ([GREEN]) → 매트릭스 한 행이 채워짐

매트릭스 빈칸 없음 = 모든 계약이 테스트·구현·커밋으로 추적된다

Dual-Track TDD 5단계 사이클과 ARRR

Track B — Logic (Entity)

- validate_lines · find_blank_coords
- Domain Mock 금지 (진짜 로직으로 검증)

Track A — UI (Boundary)

- 격자 표시 · [검증] 버튼 · E002/E003
- Domain Mock 허용 (도메인 격리)

ARRR = Ask(RED) → Respond(GREEN) → Refine(REFACTOR) → Repeat(다음 계약). 두 트랙 모두 같은 리듬, Layer만 다르다.

SESSION 3

Discovery와 하네스 구축

Mom Test로 진짜 문제를 계약으로 · R-G-I-O·PRD 고정 · 세션을 초월하는 하네스.

13:00-14:25

설계+실습

Mom Test → 제품 계약 — 발견을 검증 가능하게

원칙	Magic Square 적용
① 내 아이디어 말 안 함	"자동 검증 앱 어때요?"(금지) → "어떻게 확인했나요?"
② 과거의 구체적 행동	"마지막으로 틀렸을 때 몇 분? 어느 줄에서요?"
③ 적게 말하고 많이 듣기	솔루션(TDD·Gemini)을 먼저 팔지 않는다

증거 수준 L0(협상 불가·수치 과거행동) · L1(발화·실측) · L2(추정) — 계약마다 부착

Magic Square 계약 확정 — SSOT 계약표

ID	계약	증거	테스트
INV-1	완성+10선 34 → "pass"	L0	D-VAL-01
INV-2	완성+34아닌 줄 → "fail"+라인 ID	L0	D-VAL-02
INV-3	빈칸 있으면 "incomplete"	L1	D-VAL-03
INV-4	failed_lines $\subseteq$ 10선	L0	D-VAL-04
INV-5	find_blank_coords 1-index row-major	L1	D-LOC-01
E-1 / E-2	None→E003 / 1~16 밖→E002	L0	U-IN-01/02
UC-1	4×4 격자+[검증] 버튼→결과 표시	L2	U-VAL-01

OOS(안 할 것): 자동 풀이·채우기 · 1~16 중복 검증 — 과잉 명세는 계약이 아니라 OOS로

하네스 구축 — 세션을 초월하는 규칙 구조물

Rule

GEMINI.md

도메인·API·금지 4조항·ECB (영구 규칙)

Config

.gemini/settings.json

승인 모드·도구 통제 (push·rm 제외)

Command

.gemini/commands/*.toml

/tdd-red 등 반복 프롬프트 버튼

Skeleton

pyproject.toml · src/ · tests/

시그니처만, 본문은 계약→테스트로만

하네스 = Rule + Command + (+ Skill · Sub-Agent · Hooks) — 에이전트를 계약 안에 가둔다

금지 4조항 — 켄트 벡의 경고 신호를 규칙으로

1. 테스트 약화 금지 — assert 완화·skip·xfail·수정/삭제 금지

2. 과잉 구현 금지 — 요청한 계약 ID 외 기능 선제 구현 금지

3. 시그니처 임의 변경 금지 — 바꾸려면 Discovery로 돌아가 계약부터

4. ID 없는 동작 구현 금지 — 매트릭스에 없으면 정체불명 코드

마법상수 34 = SSOT(constants.py) · 순서 RED→GREEN→REFACTOR · 첫 줄 Phase 선언

SESSION 4

ARRR 실행 — 계약을 코드로

한 번에 계약 하나씩. red·green·refactoring 브랜치를 staging에서 갈라 머지.

14:40~16:05

실습 중심

한 계약의 여정 — RED → GREEN → REFACTOR

RED (tests/) — D-LOC-01 [INV-5]

```
def test_d_loc_01...(grid_g1):
    # Given/When/Then (AAA)
    pytest.fail(
        "RED: D-LOC-01 구현없음")

→ pytest FAILED → [RED] 커밋
```

GREEN (src/entity/)

```
MAGIC_CONSTANT=34 # [INV-1]
BLANK_CELL=0 # [INV-3]
def find_blank_coords(grid):
    return [(r+1,c+1) ...
            if cell==BLANK_CELL]
→ PASS → [GREEN] 커밋
```

실패 로그가 Context · 최소 구현만 · 구현 줄에 계약 ID 주석 · 1커밋 = RED 1묶음

Golden Master와 확장 ↔ 수축의 균형

Golden Master

- 확정 출력을 tests/golden/*.approved.txt 로 고정
- 리팩터·확장이 출력을 몰래 바꾸면 FAIL
- 전제: 대상 Test ID가 PASS 상태

확장 ↔ 수축 (Tidy First)

- 확장 2~3회마다 수축(정리) 한 번
- /refactor-smell(탐지) → /refactor-safe(1개만)
- Change Budget: 파일≤3·클래스≤1·메서드≤3

리팩터는 동작·계약 불변 — 기능 추가·버그 수정은 리팩터가 아니다(별도 GREEN)

Tidy First — 3종 커밋과 브랜치 전략

브랜치 — staging에서 갈라 머지

```
main
├─ staging      # 통합 브랜치
│  ├─ spec      # 세션3
│  ├─ red / green # 세션4
│  └─ refactoring
└─ new_features
(staging→main 은 범위 밖)
```

3종 커밋 (계약 ID 포함)

```
test(entity): D-LOC-01
              [INV-5] .. [RED]
feat(entity): D-LOC-01
              [INV-5] .. [GREEN]
refactor(entity): 헬퍼
                  추출 [REFACTOR]
```

각 단계: `git switch staging` → `switch -c <red|green|..>` → 커밋 → `merge --no-ff` → staging

SESSION 5

하루 귀환과 마무리

만든 것이 가치를 냈는가 · /export로 문서화 · KPT로 다음 회전의 가설.

16:15-17:30

문서화+회고

AARRR 계측 — 개념의 하류 귀환

만든 검증기가 실제로 가치를 냈는지 하류에서 잡니다. 계측 이벤트도 계약(M-1·M-2)으로 코드화 — Boundary 계층에서 같은 AARRR로.

폐회로 — C2C의 완성형

개념 → 코드 → 실측 → 다음 개념

예) Activation=검증 버튼 첫 클릭 · Retention=다른 판 재검증

실측이 목표 미달 → 그 수치가 다음 Discovery(Mom Test)의 질문

개념→코드→실측→다음 개념의 폐회로 — 이것이 C2C의 완성형

문서화 — /export로 세션 기록 남기기

AI 문서화 3원칙

- SSOT 인용 — PRD·GEMINI.md·git log만
- 추적 가능 — 계약 ID·커밋·테스트로
- 자동 생성 — /export → Report/NN·Prompting/NN

작업 보고서 8섹션

- 문제·목표(R-G-I-O) · 계약표 · 골든셋
- ARRR 로그 · 추적성 매트릭스 · 리팩터
- 남은 OOS·다음 후보 · KPT 회고

문서는 새 사실을 만들지 않는다 — SSOT를 인용해 추적 가능하게 남긴다

KPT 회고 — 다음 회전의 가설 만들기 (STEP 13)

Keep

계속할 것

계약→테스트→구현 순서 · 한 번
에 하나씩 · 커밋에 계약 ID

Problem

문제

막힌 지점·비용이 큰 규율·도구
마찰

Try

시도

다음 사이클에 바꿀 실험 1~2개

회고는 감상이 아니라 다음 Discovery의 입력 — Try가 다음 사이클의 가설이 된다

마 무 리

개발자 역할의 재정의

코드의 작성자에서, C2C 추적성의 설계자·감리자로.

개발자의 명시적 산출물은 계약 표, 골든셋, 실패하는 테스트, 규칙 파일, 프롬프트다.
구현 코드의 상당 부분은 AI가 생성하되 — 계약 ID·테스트·규칙이 그 품질을 게이팅한다.
AI에 위임하되 주도권을 쥐고 씨앗을 먹지 않는 것, 그것이 Augmented Coding이다.

C2C

Dual-Track TDD

ARRR

하네스

AARRR